

Machine Learning for Quant Modeling

An 8-Week Self-Study Syllabus — Foundations through Time-Series Transformers,
Anchored in Your QIP / Youtube Model / model/ Folders

Profile and constraints

Math background: strong (calculus, linear algebra, probability fluent). ML background: none. Pace: intensive, ~15–20 hrs / week over ~8 weeks. Theory depth: rigorous on the basics (linear models → backprop → sequence models), practical on transformers (read papers and code, do not re-derive). End goal: understand the *model/* folder, fix its known bugs, improve its weakest pieces, and build your own quant model that combines the strongest ideas.

How this syllabus is organized

Each week has five sections. **Theme** sets the goal. **Theory** is reading and lecture material. **Code** is hands-on work in your three folders. **Anchor** shows which existing materials in *QIP / Youtube Model / model/* this week is built around. **Deliverable** is the small thing you must produce that proves the week happened.

The general philosophy: **do not learn ML in the abstract and then come back to quant**. Every week is grounded in code from the folder. The math feeds the code, the code feeds the math, and finance feeds both. By week 8 you will have replicated and improved at least one model from *model/* end-to-end, with honest evaluation.

Time accounting. A week is roughly 16 hours: ~6 hours theory, ~8 hours coding, ~2 hours writing-up notes. If you fall behind, cut theory before cutting code; the code is what builds intuition.

Master Reading List

Buy the López de Prado. Bookmark everything else. You do not read these books cover to cover — each week of the syllabus tells you which chapters to use.

Books (primary)

- **Hastie, Tibshirani, Friedman** — *The Elements of Statistical Learning* (ESL). Free PDF at <https://hastie.su.domains/ElementsStatisticalLearning/>. Your stats / classical-ML reference. Math-dense, which suits you.
- **Goodfellow, Bengio, Courville** — *Deep Learning*. Free at deeplearningbook.org. Your deep learning reference. Read selectively — chapters 5, 6, 7, 8, 10 are the core for this syllabus.
- **Marcos López de Prado** — *Advances in Financial Machine Learning* (AFML). The single most important book for what you actually want to do. Chapters 1–7 cover financial data structures, labeling, sample weights, fractional differentiation, and (critically) cross-validation in finance.
- **Aurélien Géron** — *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). Practical complement to ESL. Use chapters 4, 6, 7, 10, 11 as readable applied versions of the more rigorous ESL chapters.

Books (secondary, optional)

- **Stevens, Antiga, Viehmann** — *Deep Learning with PyTorch*. If you want a slower PyTorch onboarding than Karpathy.
- **López de Prado** — *Machine Learning for Asset Managers*. Short, dense companion to AFML. Best chapter: denoising the covariance matrix.
- **Tsay** — *Analysis of Financial Time Series*. Classical time series (ARMA, GARCH, cointegration). Useful background for understanding why deep sequence models help and where they don't.

Papers

- **Vaswani et al. 2017** — *Attention Is All You Need* (NeurIPS). The transformer paper. You will read this in week 5.
- **Zhou et al. 2021** — *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting* (AAAI 2021, best paper). Maps directly to *model/informer/*.
- **Wu et al. 2021** — *Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting* (NeurIPS 2021). Maps directly to *model/autoformer/*.
- **Wen et al. 2022** — *Transformers in Time Series: A Survey*. The fastest way to see how Informer/Autoformer fit in the broader landscape (FEDformer, PatchTST, iTransformer, etc.).
- **Ke et al. 2017** — *LightGBM: A Highly Efficient Gradient Boosting Decision Tree* (NeurIPS). Skim only. Useful for understanding why *model/lgbm/* looks the way it does.
- **Kakushadze 2016** — *101 Formulaic Alphas* (arXiv:1601.00991). The source of the alpha101 factors your QIP project already touches.

Free online courses and articles

- **Andrej Karpathy** — *Neural Networks: Zero to Hero* (YouTube). Watch micrograd, makemore parts 1–3, and 'Let's build GPT'. These four videos alone give you almost everything you need from weeks 2 and 5.
- **3Blue1Brown** — *Neural Networks* series (YouTube). The clearest visual intuition for backprop and attention you will find.

- **Stanford CS229** — Andrew Ng's machine learning course. Lectures 1–5 and the lecture notes are the cleanest treatment of linear/logistic regression, regularization, and bias/variance.
- **Stanford CS231n** — the recurrent-network and attention lectures only. Skip the vision-specific weeks.
- **Christopher Olah** — *Understanding LSTM Networks*. colah.github.io/posts/2015-08-Understanding-LSTMs/. The single best LSTM explainer.
- **Jay Alammar** — *The Illustrated Transformer*. jalammar.github.io/illustrated-transformer/. Read this before you read Vaswani.
- **Harvard NLP** — *The Annotated Transformer*. Re-read of the Vaswani paper with line-by-line PyTorch. Open it next to *model/informer/*.
- **fast.ai** — *Practical Deep Learning*. Optional. Useful if you want a 'just train things' counterweight to the rigor.

Your own materials

- **QIP_Eng1** — pipeline framing, evaluation, IR/drawdown.
- **QIP_Eng2** — CAPM, alpha/beta, factor models. The financial vocabulary for your labels.
- **QIP_Eng3** — combination of factors, risk model, portfolio optimization. The 'what to do with *y_pred*' chapter.
- **QIP_Eng4** — genetic programming, ensemble learning, deep learning. **The single most relevant slide deck for this syllabus.**
- **QIP/data_and_models/quant_lib/** — reuse the IC/IR/group-return/drawdown evaluation harness.
- **Youtube Model/research.py & binance.py** — full data → train → eval pipeline scaffold.
- **model/** — the menu of architectures you are working toward understanding.

Week 0 — Setup and orientation (3 days, ~10 hours)

Goal: have a working Python environment, run the Youtube Model end to end at least once, and confirm that you can open and execute the files in *model/*. This is not learning ML — it is removing the friction that would otherwise leak into week 1.

Tasks

- Confirm the `.venv` already in *Quant Model/* works, or build a fresh one with Python 3.10+, PyTorch, numpy, pandas, polars, lightgbm, scikit-learn, matplotlib, altair, jupyter.
- Run *Youtube Model/binance.py* for one symbol over ~30 days. Confirm cache writes.
- Open *Youtube Model/video1.ipynb*. Run every cell top to bottom. Do not try to understand it yet — just confirm it runs.
- Open three files in *model/*: *mlp/*, *gru/*, *lgbm/*. Read the docstring at the top of each. Do not run them.
- Skim **QIP_Eng1** slides (~30 minutes). You are looking for vocabulary, not depth.

Deliverable

A scratch notebook in *Quant Model/* called *00_setup.ipynb* that loads BTCUSDT trades, aggregates to 1h OHLC bars, plots the close, and prints the dataframe shape. If you can produce that plot, week 0 is done.

Week 1 — Statistical learning foundations (rigorous)

Theme. Linear regression is the right starting point because (a) your math is strong, (b) every later model is a generalization of it, (c) your QIP code already uses it via *factor_combo_regression.py*. By the end of the week you should be able to derive ridge regression and explain why ElasticNet exists.

Theory (~6 hours)

- ESL chapters 2 and 3. Work through the math. Section 3.4 (shrinkage methods) is critical: ridge, LASSO, ElasticNet are exactly what *factor_combo_regression.py* uses.
- ESL chapter 7 sections 7.1–7.3 (bias-variance, optimism, in-sample vs out-of-sample). The most important conceptual chapter in the whole book.
- CS229 lectures 1–3 (linear regression, classification, GLMs).
- AFML chapter 7 (cross-validation in finance). **Read this twice.** Standard k-fold CV does not work on financial time series, and most ML failures in quant trace back to ignoring this chapter.

Code (~8 hours)

- Read *QIP/data_and_models/quant_lib/factor_combo_regression.py*. Identify where *LinearRegression*, *ElasticNet*, *Ridge*, *Lasso*, and *FamaMacBeth* are used. Match each to a section of ESL chapter 3.
- From scratch (no sklearn): implement OLS via the normal equations and ridge regression via the closed form. Use one of your QIP factors as *X*, next-day return as *y*.
- Replace your hand-built ridge with sklearn's. Verify identical coefficients.
- Replace simple train-test split with a **walk-forward** split. Use the technique described in AFML chapter 7 (purged k-fold with embargo).

Anchor

QIP/data_and_models/quant_lib/factor_combo_regression.py — this is the file the week is built around. **QIP_Eng2** and **Eng3** slides give the financial framing. **Youtube Model/research.py**'s

timeseries_train_test_split is the (too-naive) baseline you are improving on.

Deliverable

01_linear_models.ipynb: a notebook that runs OLS, ridge, and ElasticNet on one of your alpha101 factors against next-day returns, with proper purged walk-forward CV, and reports IC, IR, and a long-short equity curve. By the end you should know exactly what each *alpha* and *l1_ratio* mean.

Week 2 — Neural network fundamentals (rigorous)

Theme. The MLP is the building block of every model in *model/*. Every architecture in that folder ends with a 4-layer MLP head. By the end of this week you should be able to derive backprop on paper for a 2-layer network and explain why ReLU + Adam + dropout became the default cocktail.

Theory (~7 hours)

- Goodfellow chapter 5 (ML basics) — skim, you saw most of this in week 1.
- Goodfellow chapter 6 (deep feedforward networks) — **read carefully**. Understand the universal approximation theorem at a hand-wave level, the architecture choice rationale, and the role of nonlinearity.
- Goodfellow chapter 7 (regularization) — sections 7.1, 7.4, 7.5, 7.8, 7.12 (dropout) are required.
- Goodfellow chapter 8 (optimization) — SGD, momentum, Adam. Skim 8.5 (init).
- Karpathy *Zero to Hero*: micrograd (60 min) and makemore part 1 (75 min). Watching these is non-negotiable. Karpathy's micrograd is the single best backprop derivation in existence.
- 3Blue1Brown's neural network series (4 episodes, ~70 min total).

Code (~8 hours)

- Implement an MLP from scratch in numpy on synthetic regression data. Forward pass, manual backprop, SGD. No PyTorch.
- Re-implement the same MLP in PyTorch, training on the same data. Confirm matching loss curves.
- Open *model/mlp/mlp_model_explained.py*. Walk through it line by line. Note the exact architecture: $num_i \rightarrow num_h \rightarrow num_{h/2} \rightarrow num_{h/4} \rightarrow num_o$, with ReLU + dropout after each. This pattern repeats in every other model in the folder.
- Replace the *NonLinearModel* in *Youtube Model/models.py* with the MLP from *model/mlp/*. Train it with *research.py*'s *batch_train_reg*. Compare the test sharpe to the original.

Anchor

model/mlp/mlp_model_explained.py. Also **Youtube Model/models.py** and **research.py**'s *batch_train_reg*. QIP_Eng4 has a deep-learning section that gives the financial framing.

Deliverable

02_mlp_from_scratch.ipynb: a notebook that (1) trains a numpy MLP on synthetic data, (2) trains an equivalent PyTorch MLP, (3) trains *model/mlp/Model* on real BTCUSDT log returns via the Youtube pipeline, with honest walk-forward eval. Plus a one-page handwritten or markdown derivation of backprop for a 2-layer network.

Week 3 — Sequence models (rigorous)

Theme. RNNs, LSTMs, GRUs, and 1D convolutions. By the end of this week you understand why sequential information matters, why vanilla RNNs fail, and how the gating in LSTM/GRU fixes it. This is the longest code week — six files in *model/* are touched.

Theory (~7 hours)

- Goodfellow chapter 10 (sequence models). Sections 10.1, 10.2, 10.7, 10.10 are the core. 10.7 (long-term dependencies / vanishing gradient) is the motivation for LSTM/GRU.
- Colah's *Understanding LSTM Networks*. Read it twice. Most people who 'know LSTMs' only know them at this level — that level is enough.

- Karpathy's makemore parts 2–3 (RNN, then character-level transformer). Part 2 alone gives you a working RNN intuition in PyTorch.
- CS231n lecture on RNNs (one video, ~70 min).
- Skim: a 1D convolution explainer (any well-written blog post). The mechanic is identical to 2D conv but on time instead of pixels.

Code (~9 hours)

- Read *model/gru/gru_model_explained.py*. Identify the GRU layer, the *all_steps* flag, and the MLP head — same head as week 2.
- Read *model/cnn/multiscale_cnn_explained.py*. Note the three branches at different horizons. Sketch the tensor-shape progression on paper.
- Read *model/seq/two_stage_gru_mlp_explained.py*. This is GRU + last-step-MLP, predictions added. A simple but real ensemble pattern.
- Read *model/mlpop/rolling_feature_mlp_explained.py*. The handcrafted rolling-feature operators (Mean, Std, Min, Max, Mean_res) are the most interesting part. **Note the bugs** (dropout applied to *x* not *out*, unused modules) — fix them as an exercise.
- Read *model/nec/nec_hybrid_explained.py*. The LSTM → classifier → routing-to-regressor pattern. **Note the inconsistency** — 3-class softmax but binary routing — and decide what you would do instead.
- Wire *model/gru/Model* into the Youtube pipeline. Build a sliding-window dataloader so the GRU sees sequences, not single rows. This is the first real engineering challenge of the syllabus.

Anchor

Five files in model/: *gru/*, *cnn/*, *seq/*, *mlpop/*, *nec/*. QIP_Eng4 again, the deep-learning section. Youtube Model video3.ipynb shows the streaming-window pattern you need to replicate.

Deliverable

03_sequence_models.ipynb: a notebook that trains *model/gru/Model* on BTCUSDT 1h log-return forecasting using a 24-step lookback. Plus a markdown writeup answering: (a) why does GRU outperform / underperform the MLP from week 2? (b) what is the dropout bug in *mlpop/* and how would you fix it? (c) what is the routing inconsistency in *nec/* and what would replace it?

Week 4 — Tree models and the quant pipeline (deep dive)

Theme. The week you reverse-engineer *model/lgbm/lgbm_quant_pipeline_explained.py* completely. This is the only file in *model/* with a full production pipeline, and almost every pattern in it (parallel parquet IO, label bucketing, +/- balancing, incremental training, IC monitoring) generalizes to any model. Plus you learn the actual model class — gradient boosting — that powers it.

Theory (~6 hours)

- ESL chapters 9.2 (CART) and 10 (boosting). Chapter 10 is the conceptual core: AdaBoost, gradient boosting, and the loss-function view of boosting.
- Hands-On ML chapter 7 (ensemble learning, random forests, boosting). Practical complement to ESL Ch 10.
- LightGBM paper (Ke et al. 2017) — skim only. You care about histogram-based splitting, leaf-wise growth, and GOSS. 10 minutes of skimming, not a full read.
- AFML chapters 1–5 carefully: financial data structures, labeling (the triple-barrier method), meta-labeling, sample weights, fractional differentiation. **This is the most important reading of the syllabus** if you actually want to do quant ML rather than just ML.

Code (~10 hours)

- Read *model/lgbm/lgbm_quant_pipeline_explained.py* top to bottom. It is 1,109 lines. Schedule two ~3-hour sessions for it.
- Annotate the file in your own words. For each function (*read_data*, *parallel_read_data*, *create_labels*, *downsample_zero_label*, *balance_labels*, *run_fixed_threshold_training*, *run_quantile_threshold_training*), write a 2-sentence comment in your notebook explaining what it does.
- Re-implement *create_labels* and *balance_labels* on a small toy series. You should be able to explain what 'signed magnitude bucket labels' means without looking.
- Build a smaller LightGBM pipeline on BTCUSDT data using the patterns from this file. Inputs: rolling features (mean, std, return lags). Output: bucketed next-period log return. Track IC during training using the callback pattern.
- Compare your LightGBM model's IC and sharpe against the GRU from week 3 and the MLP from week 2.

Anchor

model/lgbm/lgbm_quant_pipeline_explained.py — the entire week is built around this one file. QIP_Eng4 (ensemble learning section). AFML chapters 1–5.

Deliverable

04_lgbm_pipeline.ipynb: a working scaled-down LightGBM pipeline that uses signed-bucket labels, balanced classes, and an in-training IC callback. Plus an annotated copy of *lgbm_quant_pipeline_explained.py* with your own comments next to each function. By the end you should be able to explain to someone else, in 5 minutes, why this file is structured the way it is.

Week 5 — Attention and transformers (practical)

Theme. Now we shift to practical depth. You read papers, you read annotated code, but you do not re-derive multi-head attention from scratch. By the end of the week you can read the encoder/decoder block of any transformer paper and follow the data flow.

Theory (~7 hours)

- **Read in this order:** (1) Jay Alammar's *Illustrated Transformer*, (2) the original Vaswani *Attention Is All You Need* paper, (3) Harvard NLP's *Annotated Transformer*.
- Karpathy's *Let's build GPT from scratch* video (~2 hours). This is the single best practical introduction to attention. You write a working transformer in PyTorch alongside the video.
- Goodfellow chapter 12 sections 12.4 (attention mechanisms) — light skim.
- Optional: *The Illustrated GPT-2* by Jay Alammar — gives a mental model of decoder-only vs encoder-decoder transformers.

Code (~8 hours)

- Code along with Karpathy's *Let's build GPT* video. Produce a working tiny transformer.
- Modify the tiny transformer to do regression instead of next-token prediction, on synthetic time-series data. This is the architectural shape you will see in Informer/Autoformer next week.
- Pre-read `model/informer/informer_explained.py` and `model/autoformer/autoformer_explained.py`. Do not try to understand them yet — just count the layers, identify the encoder/decoder split, and find the embedding, attention, and feed-forward blocks.

Anchor

Vaswani 2017. Karpathy's GPT video. Pre-reading of the two transformer files in `model/`.

Deliverable

`05_tiny_transformer.ipynb`: a working transformer regressor on synthetic time-series data. Plus a one-page diagram (hand-drawn or in any tool) showing how a single transformer block transforms an input (B, L, D) tensor through Q/K/V projection, multi-head attention, skip connection, layer norm, feed-forward, skip, layer norm.

Week 6 — Time-series transformers (practical)

Theme. Read the Informer and Autoformer papers, then read the corresponding files in *model/* alongside them. Both papers exist because vanilla attention is $O(L^2)$ in sequence length, which is fatal for the long lookbacks you want in finance. Each paper has a different fix — you should be able to explain both.

Theory (~7 hours)

- Wen et al. 2022 *Transformers in Time Series: A Survey* — read sections 1–3. This gives you the landscape: where Informer / Autoformer / FEDformer / PatchTST sit relative to each other.
- Zhou et al. 2021 (**Informer**) — full paper. Pay attention to: (1) the ProbSparse self-attention idea (sample top queries by KL divergence), (2) the self-attention distillation between encoder layers, (3) the generative-style decoder for one-shot prediction.
- Wu et al. 2021 (**Autoformer**) — full paper. Pay attention to: (1) series decomposition (trend + seasonal), (2) Auto-Correlation in place of attention, computed via FFT, (3) the time-delay aggregation step.
- Optional — FEDformer or PatchTST paper, just to see how the field evolved after Autoformer.

Code (~9 hours)

- Open *model/informer/informer_explained.py* next to the Informer paper. Map each class (*ProbAttention*, *AttentionLayer*, *ConvLayer*, *EncoderLayer*, *Encoder*, *DecoderLayer*, *Decoder*, *Informer*, *InformerEncOnly*) to a section in the paper.
- Open *model/autoformer/autoformer_explained.py* next to the Autoformer paper. Map each class (*AutoCorrelation*, *AutoCorrelationLayer*, *EncoderLayer*, *DecoderLayer*, *Encoder*, *Decoder*, *Autoformer*) to a section in the paper. Pay extra attention to the FFT-based *time_delay_agg_training* function.
- Run *InformerEncOnly* on a sequence-prediction task on BTCUSDT data. (The encoder-decoder version is harder to wire up — skip it unless you have time.)
- Confirm the suspicious decoder branch flagged in *autoformer_explained.py*'s docstring. Decide whether the bug needs fixing for your use case.

Anchor

model/informer/ and **model/autoformer/** — the entire week. Plus the two papers.

Deliverable

06_ts_transformers.md: a 2-page writeup answering, for each of Informer and Autoformer, (a) what specific problem with vanilla attention does it solve, (b) what is the core mechanism that solves it, (c) where in the corresponding *_explained.py* file is that mechanism implemented, and (d) for which kinds of financial targets would you reach for it. Plus *06_informer_run.ipynb* with *InformerEncOnly* trained on BTCUSDT.

Week 7 — Synthesis and honest backtesting

Theme. No new theory. You combine the pipeline from *Youtube Model*, the evaluation discipline from *QIP/quant_lib/*, and at least three architectures from *model/* into a single comparison framework. The goal is one notebook that runs the same data through different models and reports the same set of honest metrics for each.

Theory (~3 hours)

- AFML chapters 11 (the dangers of backtesting) and 12 (deflated sharpe ratio). **Required.** Most published quant ML results are wrong because they ignore these chapters.
- Skim `QIP/data_and_models/quant_lib/factor.py` to absorb the IC / IR / group-return / drawdown evaluation patterns.

Code (~12 hours)

- Build a unified evaluation function. Inputs: `y_true`, `y_pred`. Outputs: IC (Pearson and Spearman), IR, sharpe, max drawdown, hit rate, turnover, sharpe net of fees. Use the formulas from `QIP/quant_lib/factor.py` as the reference.
- Build a unified data loader. Same dataset, same train/val/test split (purged walk-forward), fed to every model.
- Run at least four models on the same task: (1) ridge regression baseline, (2) MLP (week 2), (3) GRU (week 3), (4) LightGBM (week 4). Optional fifth: *InformerEncOnly* from week 6.
- Produce a comparison table: model × metric. Write one paragraph for each model honestly explaining where it does well and badly.
- Fix the dropout bug in `model/mlpop/` and the routing inconsistency in `model/nec/`. Add fixed versions to your comparison.

Anchor

All three folders. `QIP/quant_lib/factor.py` for evaluation, *Youtube Model/research.py* for the training loop, `model/` for architectures.

Deliverable

`07_model_comparison.ipynb`: one notebook, one dataset, one evaluation function, five trained models, one summary table. The table should fit on one screen. If the comparison is honest, the simplest model often wins — that is the lesson.

Week 8 — Capstone: build your own model

Theme. One week to design, train, and honestly evaluate your own quant model. Not a research paper, not a new architecture — a defensible combination of ideas you have already learned, with a clean evaluation. The goal is not to beat the existing models on every metric. The goal is to *understand exactly why your model performs the way it does*.

Recommended starting design

Pick a target you can defend financially: BTCUSDT 8-hour log return is the default suggested by the Youtube videos and is a sane choice. Build features from rolling statistics on lagged log returns, rolling volatility, and simple volume-based factors. Use the rolling operators from *model/mlpop/* (with the dropout bug fixed) as feature extractors. Feed those features into a GRU + last-step MLP additive ensemble (the pattern from *model/seq/*). Train with the LightGBM-style label bucketing and class balancing (week 4). Evaluate with your unified harness from week 7. Compare to a ridge regression baseline. If you cannot beat ridge net of fees, the architecture is not the problem.

Suggested improvements over the existing models

- **Fix nec's routing.** Replace hard-argmax routing with a soft mixture: $pred = \sum_k p_k * regressor_k(x)$. Compare hard vs soft on the same data.
- **Fix mlpop's dropout.** Apply it to the activations, not the input.
- **Add purged walk-forward CV to lgbm.** The existing pipeline uses a single split; AFML chapter 7 gives you the right thing.
- **Add a deflated sharpe ratio** to the evaluation. AFML chapter 12.
- **Use GP-discovered factors** from your QIP project as additional features. Bridges the QIP folder into the model folder.
- **Try transaction-cost-aware loss.** Penalize predictions that imply high turnover. Often more impactful than architecture changes.

Code

- *08_capstone.ipynb*: data loading, feature engineering, model definition, training, evaluation.
- *08_capstone_writeup.md* (or a docx if you prefer): (1) target choice and economic justification, (2) feature design and why each feature, (3) architecture choice and what it is supposed to capture, (4) training procedure (loss, optimizer, schedule, validation), (5) results vs baselines (table + equity curves), (6) ablation: what happens if you remove rolling features? if you replace GRU with MLP? if you remove balancing? (7) honest failure modes and what you would try next.

Deliverable and finish line

If at the end of week 8 you have a model you can explain end-to-end, evaluation you trust, a concrete list of the things in *model/* that you fixed or improved, and a clear sense of what would be next, the syllabus is finished. You will not be done learning — ML is too big — but you will be ahead of most people who claim to do quant ML, because you can defend every line of your pipeline.

Appendix — Things to keep visible all 8 weeks

The four hard truths from every *_explained.py* file

- The architecture is not the edge. Feature design and label design are.
- Standard k-fold CV is broken on financial time series. Use purged walk-forward with embargo.

- Most strategies that look profitable in a backtest are dead in production. Transaction costs and slippage are not optional.
- Regime stability is the silent killer. A model that works on 2024 data may be useless in 2026.

Recurring metrics you should always compute

- **IC** (Information Coefficient): Pearson correlation between prediction and realized return.
- **Rank IC**: Spearman correlation. More robust than IC.
- **IR** (Information Ratio): mean / std of strategy return, often annualized.
- **Sharpe (net of fees)**: do this with realistic taker / maker fee assumptions.
- **Max drawdown**: peak-to-trough loss along the equity curve.
- **Hit rate**: fraction of correctly-signed predictions.
- **Turnover**: how much of the portfolio changes between rebalances. High turnover kills sharpe under fees.
- **Deflated sharpe ratio** (AFML Ch 12): sharpe corrected for the number of trials in your search.

If you fall behind

The compressible weeks are 5 and 6. You can survive on Karpathy's GPT video plus the Illustrated Transformer for week 5, and the two transformer papers' abstracts plus the diagrams in the *_explained.py* files for week 6. The non-compressible weeks are 1, 2, 4, 7. If you have to skip one, skip 6. If you have to skip two, skip 5 and 6 — transformers are popular but tree models still win in production more often than people admit.